# Naked Objects: Gemini Rewrite

## From Domain Models to Complete Applications

Anonymous Author(s)


## Abstract

The **Naked Objects** architectural pattern proposes that complete user interfaces can be derived automatically from domain models. Instead of manually maintaining separate models, component trees, form layouts, validators, persistence mappings, localization files, and AI tool specs, developers declare their domain logic once and allow the runtime engine to project it into every required form.

While early Naked Objects systems successfully demonstrated this derivation, they relied on a rigid, primitive grammar—rendering objects as forms and collections as tables. This created a severe "usability gap" that restricted Naked Objects to internal tools and administrative backends.

This paper presents a modern, flexible interaction grammar that closes this usability gap without requiring hand-authored UI component trees. We show that most interactions in structured-information software reduce to two core operations: (1) **traversing relationships** across an object graph via recursive, orientation-flexible master-detail views (Miller columns), and (2) **projecting collections** along semantic dimensions (such as linear feeds, inline documents, anchored tabs, and aligned matrices).

We demonstrate **Strvct**, an open-source client-side framework implementing this model-driven approach. By centralizing the model-to-view pipeline through slot metadata annotations, Strvct delivers architectural capabilities "for free": automatic responsive collapse, headless testability, transparent persistence and cloud synchronization, unified accessibility, and a natively AI-operable interface with a constant-size tool surface.

In a production case study (**undreamedof.ai**, an AI tabletop gaming platform with ~90 domain classes), ~98% of all views are framework-generated. The remaining custom views are inherently graphical media (a 3D dice engine and a battle map), confirming that the boundary of automatic UI generation falls naturally where spatial geometry carries irreducible domain meaning.



***CCS Concepts:*** • **Software and its engineering** → **Object oriented frameworks**; *Model-driven software engineering*; • **Human-centered computing** → *Interactive systems and tools.*

***Keywords:*** naked objects, automatic UI generation, model-driven development, semantic projection, composable primitives, AI agents, Miller columns


## 1 Introduction

Modern software development spends immense effort creating and maintaining user interfaces. In traditional component frameworks (such as React, Vue, or Svelte), every screen is authored as a bespoke artifact. When a data model changes—such as adding a field to a customer profile—the developer must manually update:

- The domain model class
- Form UI layouts and input fields
- Client-side and server-side validation rules
- Persistence schemas and ORM mappings
- Navigation graphs and routing states
- Localization string dictionaries
- Accessibility ARIA attributes
- AI agent tool definitions and API endpoints

This structural duplication creates exponential maintenance costs and frequent synchronization bugs.

```
Traditional Component Approach (Manual Propagation):
Domain Model -> UI Components -> Forms -> Validators -> Per

Model-First Naked Objects Approach (Automatic Projection):
                +-> Auto-Generated UI (Master-Detail, Tabs
                |-> Auto-Persistence & Cloud Sync
Annotated Model -+-> Headless Execution & Fast Testing
                |-> Native AI Tool Surface (Constant Size)
                +-> Automatic Accessibility & Localization
```

**Naked Objects**, introduced by Pawson and Matthews in 2002 [3], proposed a radical alternative: expose domain objects directly to users and generate the entire interface automatically. Because there is no separate UI code, consistency is guaranteed—the interface always perfectly reflects the live state of the domain.

Despite its obvious maintenance benefits, early Naked Objects implementations (such as Apache Isis/Causeway [**?** ]) struggled with widespread adoption. They derived every object into a form and every collection into a table. While functionally complete, these interfaces felt clunky and rigid. They lacked spatial context, viewport adaptability, depth indicators, and visual continuity.

This paper shows how to close that usability gap. We contend that the design space for information management software is remarkably narrow and structured. By replacing generic forms and tables with a richer grammar—**recursive relationship traversal** and **semantic collection projection**—frameworks can derive modern, beautiful, and responsive interfaces directly from annotated models.

## 2 The Usability Gap in Prior Systems

To understand why classic Naked Objects stalled, we must examine the specific usability limitations of early form-and-table systems:

1. **Lack of Spatial Hierarchy:** Generic forms flatten relationships into lists of text boxes and dropdowns. Users cannot visually perceive ownership, containment, or structural depth.
2. **Poor Viewport Adaptability:** Form-based tools are typically designed for fixed desktop resolutions. Adapting them to mobile screens requires ad-hoc, per-screen responsive hacks.
3. **Loss of Context During Navigation:** Drilling into a child object in a form interface usually replaces the screen or pops up a modal window, disorienting the user.
4. **Disjointed Mental Models:** Without spatial continuity, users treat each screen as a disconnected island rather than exploring a unified object graph.

Component frameworks (React, Vue) address this problem by giving developers total visual freedom, but at the cost of authoring every screen by hand. Our goal is to achieve the visual quality and spatial legibility of hand-crafted apps while keeping the zero-UI-maintenance benefits of Naked Objects.

## 3 A Narrow Design Space for Information Apps

A common misconception is that auto-generation forces all apps to look like boring administrative dashboards. However, a deeper analysis of consumer and enterprise software reveals a powerful pattern:

> **Most interactions in structured-information applications consist of traversing relationships among domain objects and projecting collections of those objects along semantic dimensions.**

Consider CRMs, project management tools, email clients, social networks, healthcare records, e-commerce stores, and settings panels. Although their visual styling differs, their fundamental operations are identical: users find an object, follow a relationship, inspect or edit details, and organize collections.

```
Typical Domain Graph:
Workspace
+-- Projects
|   +-- Project
|       +-- Tasks
|       +-- People
|       +-- Milestones
+-- Customers
+-- Documents
```

### 3.1 Relationship Traversal vs. Collection Projection

We divide the user interface grammar into two distinct layers:

- **Relationship Traversal:** Moving along links in the object graph (e.g., from a *Project* to its *Tasks*, then to a specific *Task*).
- **Collection Projection:** Displaying a set of related objects in a specific semantic layout (e.g., viewing *Tasks* as an ordered list, a Kanban board grouped by status, a table, or a timeline).

### 3.2 The Boundary of Automatic Generation

Where does automatic generation end and custom code begin? The boundary is clean and objective:

- **Generated UI (Domain Model):** Managing structured data where spatial layout is purely an organizational aid (text, relationships, properties, categories, dates).
- **Custom UI (Specialized Media):** Surfaces where spatial position carries *irreducible domain meaning*.

Examples of surfaces requiring custom UI renderers:

- **Creative tools:** Canvas drawing, code editors, rich audio/video sequence editors.
- **Graphical visualizations:** 3D game viewports, interactive geographic maps, medical imaging scans.
- **Data plots:** Complex financial or scientific charts.

For a CRM or settings panel, generated UI covers ~100% of the app. For an IDE or game, generated UI handles the application shell (file trees, properties, settings, inventory), while a specialized custom renderer powers the central canvas.

## 4 Architecture & UI Mechanics in Strvct

Strvct implements this two-part grammar using client-side JavaScript. The core visual building blocks are **Tiles**, **Tile Stacks**, and **Master-Detail Views**.

### 4.1 Tiles and Tile Stacks

The fundamental primitive in Strvct is the **Tile**:

- **Summary Tile:** Displays a domain object with a title, subtitle, icon, and contextual actions. Selecting a summary tile opens its details in an adjacent column.

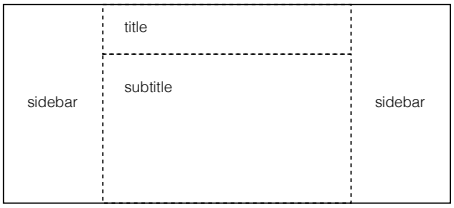



**Figure 1.** Summary Tile Primitive.

| Mechanism | Semantic mapping | Familiar UI Presets |
|---|---|---|
| Recursive composition | Parent-child relationships or derived groupings | Navigation tree, grouped list, Kanban board |
| Cross-sibling alignment | Equivalent property slots aligned across items | Data table, spreadsheet, pivot matrix |
| Value-to-space mapping | Numeric, temporal, or spatial values mapped to position/extent | Timeline, Gantt chart, calendar, map |
| Quantitative encoding | Numeric values mapped to visual marks or axes | Line chart, bar plot, heatmap |

**Table 1.** Projection mechanisms and the familiar UI forms they yield.

- **Property Tile:** Displays an individual object property (key-value pair) with direct inline editing (text inputs, numbers, toggles, date pickers) and validation feedback.

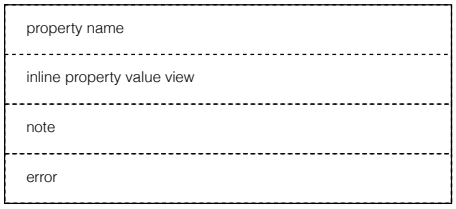



**Figure 2.** Property Tile Primitive.

A **Tile Stack** is a scrollable, ordered list or grid of tiles. Stacks support drag-and-drop reordering, slide-to-delete, and direct data import/export.

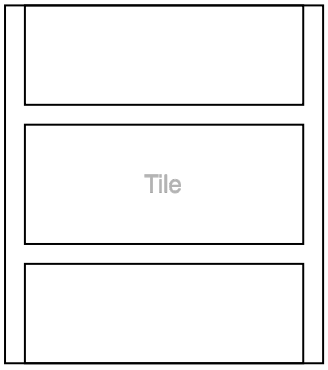



**Figure 3.** Tile Stack Primitive.

### 4.2 Recursive Master-Detail Navigation

A **Master-Detail View** pairs a master tile stack with a detail pane showing the selected object's contents. Because detail panes can themselves contain child master-detail views, navigation composes recursively to arbitrary depth.

Key characteristics of Strvct's master-detail engine:

- **Flexible Orientation:** Each level can layout horizontally (like macOS Miller column file browsers [2]) or vertically (like nested accordion lists or settings panels).

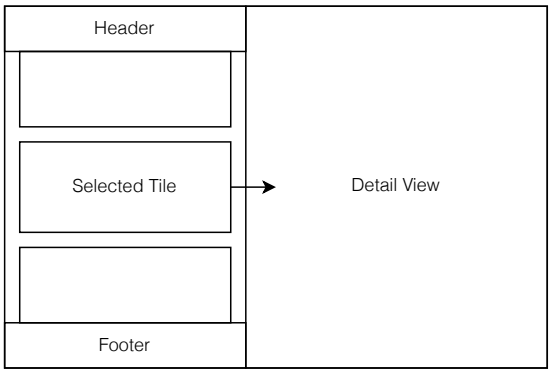



**Figure 4.** Master-Detail Navigation View.

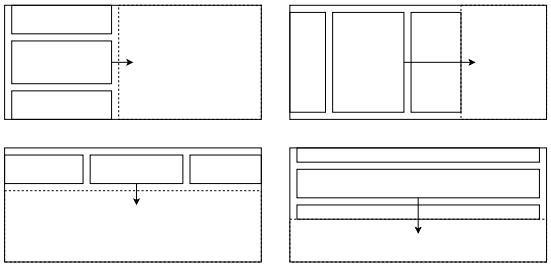


**Figure 5.** Master-Detail Orientations.

- **Automatic Mobile Collapse:** When viewport space is constrained, earlier columns automatically collapse into a mobile push/pop stack equipped with interactive breadcrumbs. Zero per-screen responsive design code is required.

### 4.3 Advanced Inline Layouts

While column traversal is the default, certain complex screens require rendering child views directly inside parent tiles rather than pushing new columns. Strvct provides three built-in inline placement policies:

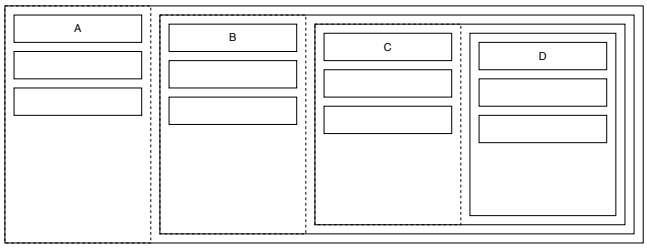



**Figure 6.** Expanded Viewport (Desktop Layout).

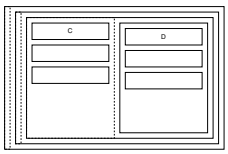



**Figure 7.** Collapsed Viewport (Mobile Layout with Breadcrumbs).

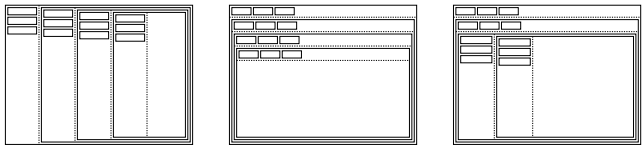


**Figure 8.** Miller Column Nesting Patterns: Vertical (left), Horizontal (center), Hybrid (right).

1. **Anchored Tabs:** Renders selected children as interactive side-by-side panes or tab bars. Width arbitration automatically fits as many panes as space permits on wide desktop monitors while gracefully degrading to standard tabs on mobile.
2. **Document Inlining:** Renders a nested subtree as a continuous multi-column document with flowing sections, headers, and fields. Fields edit in place without triggering navigation transitions.
3. **Presented Nodes:** Allows an object to delegate its presentation to a derived node (e.g., displaying a structured RPG character sheet instead of raw property slots), providing bespoke reading views without custom component templates.

## 5 Model Declarations & The Annotation Bridge

Developers define domain models as JavaScript classes inheriting from `SvStorableNode`. Properties are declared as **slots** enriched with metadata annotations:

```
(class Character extends SvStorableNode {

    initPrototypeSlots () {
        // Name slot: String property, persisted, synced to view, editable
        {
            const slot = this.newSlot("name", "");
            slot.setSlotType("String");
            slot.setShouldStoreSlot(true);
            slot.setSyncsToView(true);
            slot.setCanEditInspection(true);
        }
        // Level slot: Number property
        {
            const slot = this.newSlot("level", 1);
            slot.setSlotType("Number");
            slot.setShouldStoreSlot(true);
            slot.setSyncsToView(true);
        }
        // Inventory relationship: Navigable child object
        {
            const slot = this.newSlot("inventory", null);
            slot.setFinalInitProto(Inventory);
            slot.setIsSubnodeField(true);
        }
    }

    subtitle () {
        return "Level " + this.level();
    }

}.initThisClass());
```

### 5.1 The Multi-Reader Annotation Architecture

Slot annotations serve as a decoupling bridge. Each framework subsystem reads slot metadata independently without cross-system dependencies:

- **UI Layer:** Reads `slotType`, `canEditInspection`, and `isSubnodeField` to instantiate Property Tiles and navigate child subnodes.
- **Persistence Layer:** Reads `shouldStoreSlot` to serialize records into local storage (IndexedDB) and cloud object pools.
- **Sync Scheduler:** Reads `syncsToView` to observe slot updates and post coalesced, deferred notifications to active views.
- **AI Agent Surface:** Reads property types and constraints to auto-generate JSON validation schemas and patch tools.
- **Localization Engine:** Reads property names and translation contexts to auto-translate labels.

```
                    +-> UI Rendering Layer (Property Tiles &
                    |-> Persistence Engine (IndexedDB & Cloud
Slot Metadata ----+-> Sync Scheduler (Weak Observer Notific
                    |-> AI Agent Infrastructure (Patch Schema
                    +-> Internationalization Engine (Label Tr
```

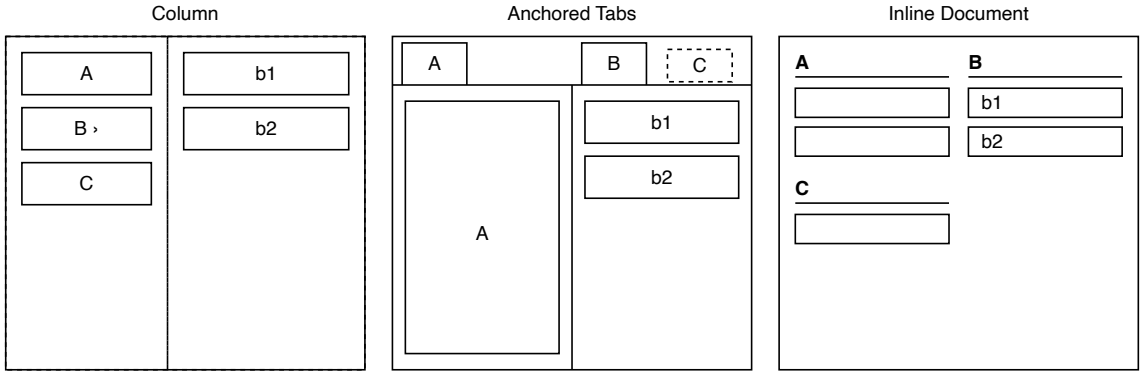



**Figure 9.** Comparison of Subtree Layouts: Columns, Anchored Tabs, and Inline Document Flow.

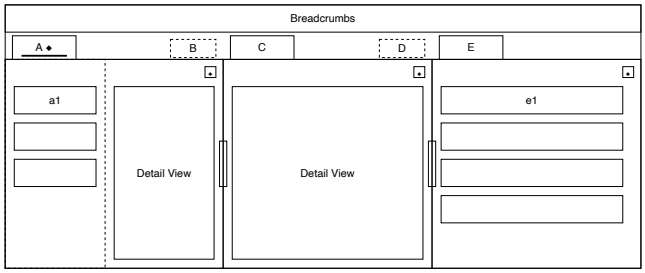



**Figure 10.** Anchored Tabs (Wide Desktop Viewport).

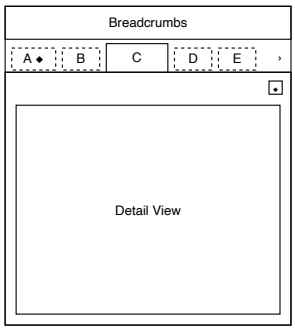



**Figure 11.** Anchored Tabs (Narrow Mobile Viewport).

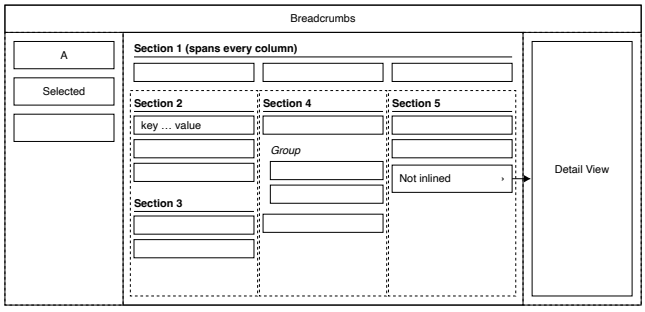



**Figure 12.** Inline Document Section Flow.

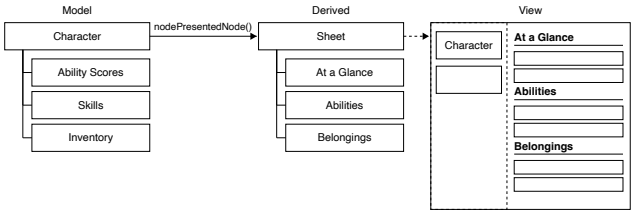



**Figure 13.** Presented Node Pattern.

## 6 Structural Consequences ("Free" Capabilities)

Centralizing the model-to-view pipeline through slot annotations yields powerful architectural capabilities that traditional component frameworks must build and maintain manually for every screen:

### 6.1 Natively AI-Operable Model

Because slot metadata describes every class, Strvct exposes a universal AI tool surface consisting of just two generic functions:

1. `schema-fetch`: Retrieves the slot schema for any domain class or object instance.
2. `apply-patch`: Applies a JSON patch payload to update properties or mutate child relationships.

This tool surface remains **constant in size** regardless of whether the application contains 5 domain classes or 500. When an AI agent generates an invalid field patch, the rejected response includes the slot's schema, allowing the model to self-correct in real time.

### 6.2 Headless Execution and Ultra-Fast Testing

Domain nodes contain zero DOM or browser dependencies. The exact same domain classes run unchanged under Node.js. Unit and integration tests instantiate model graphs, invoke domain methods, and assert against slot states at lightning speed without spinning up headless browser instances (Puppeteer/Playwright).

### 6.3 Transparent Persistence and Cloud Sync

Strvct divides persistence into two unified strategies:

- **Local Object Pools:** Fast, synchronous IndexedDB storage for domain object state.
- **Content-Addressable Blob Store:** Asynchronous SHA-256 keyed storage for large binary assets (images, audio, files).

Cloud synchronization sends versioned atomic commits containing only the modified slot records. Conflicts are detected deterministically per document pool.

### 6.4 Universal Accessibility and Localization

Accessibility and internationalization are handled at the framework primitive level. Because every view is rendered by ~15 shared view classes, ARIA roles, focus management, keyboard navigation, and translation hooks are paid once. Fixing an accessibility or translation bug in a primitive tile fixes it across every screen in the entire application.

### 6.5 Live Inspector & Developer Mode

Every tile supports modifier-clicking (Option + Click) to reveal a live Property Inspector displaying all declared slots, storage flags, and validation states. Developers can toggle Developer Mode to inspect hidden internal subnodes without writing custom debugging views.

## 7 Empirical Case Study: undreamedof.ai

Strvct was used to build **undreamedof.ai**, a commercial AI-powered virtual tabletop application for tabletop roleplaying games (D&D 5e).

| Subsystem | Domain classes | Custom views |
|---|---|---|
| Character system | ~30 | 0 |
| Campaign hierarchy | ~20 | 1 (Map Viewer) |
| Session & Gameplay | ~25 | 1 (3D Dice Engine) |
| AI Integration & Tools | ~15 | 0 |
| **Total** | **~90** | **2** |

**Table 2.** undreamedof.ai breakdown by subsystem.

Over 98% of all application screens—character sheets, inventory managers, spellbooks, campaign trees, settings panels, chat logs, and admin inspectors—are derived automatically by Strvct.

The only two custom views in the entire application are:

1. A **3D WebGL Dice Roller**
2. An **Interactive Tactical Battle Map**

Both represent specialized graphical media where spatial geometry carries irreducible domain meaning (§3). Their presence confirms that Strvct's boundary between auto-generated and custom UI falls exactly where predicted.

## 8 Comparison with Related Paradigms

## 9 Limitations & Future Work

While Strvct successfully demonstrates the power of modern Naked Objects, several open challenges and limitations remain:

1. **Inherently Graphical Media:** Applications centered on canvas editing, CAD, video sequencing, or 3D gaming will always require custom view renderers.
2. **Ecosystem Isolation:** To support content-addressable bundling and zero-build client loading, Strvct uses a custom resource loader rather than standard npm/Webpack/Vite tooling.
3. **Aligned Grid Projections (Tables & Spreadsheets):** While inline document flow and anchored tabs are production-proven, cross-sibling column alignment (making row tiles align into full spreadsheets) is currently a proposed framework extension.
4. **Extreme Collection Scaling:** Lazy view instantiation keeps rendering lightweight, but virtualized viewport scrolling for mega-collections ($10^5$+ items) remains an active area of optimization.

## 10 Conclusion

For twenty-five years, the Naked Objects pattern offered an enticing promise: write the domain model, and the application will follow. Early implementations stalled not because derivation was impossible, but because forms and tables were too primitive to deliver modern user experiences.

Strvct solves this by establishing a complete, flexible interaction grammar based on **recursive relationship traversal** and **semantic collection projection**. By eliminating manual UI component authoring, developers eliminate view-layer maintenance, reduce synchronization bugs, and gain native AI operability, instant headless testing, transparent persistence, and universal accessibility.

As AI agents become routine co-processors of software state, the cost of maintaining separate models, UI components, and tool specs will become unsustainable. Model-first derived architectures offer a durable, elegant alternative: software where the user interface is a live, faithful projection of the domain model.

## References

[1] ]causeway Apache Software Foundation. [n. d.]. Apache Causeway. https://causeway.apache.org/ Formerly Apache Isis.
[2] Wikipedia contributors. 2024. Miller columns. https://en.wikipedia.org/wiki/Miller_columns
[3] Richard Pawson and Robert Matthews. 2002. *Naked Objects.* John Wiley & Sons.

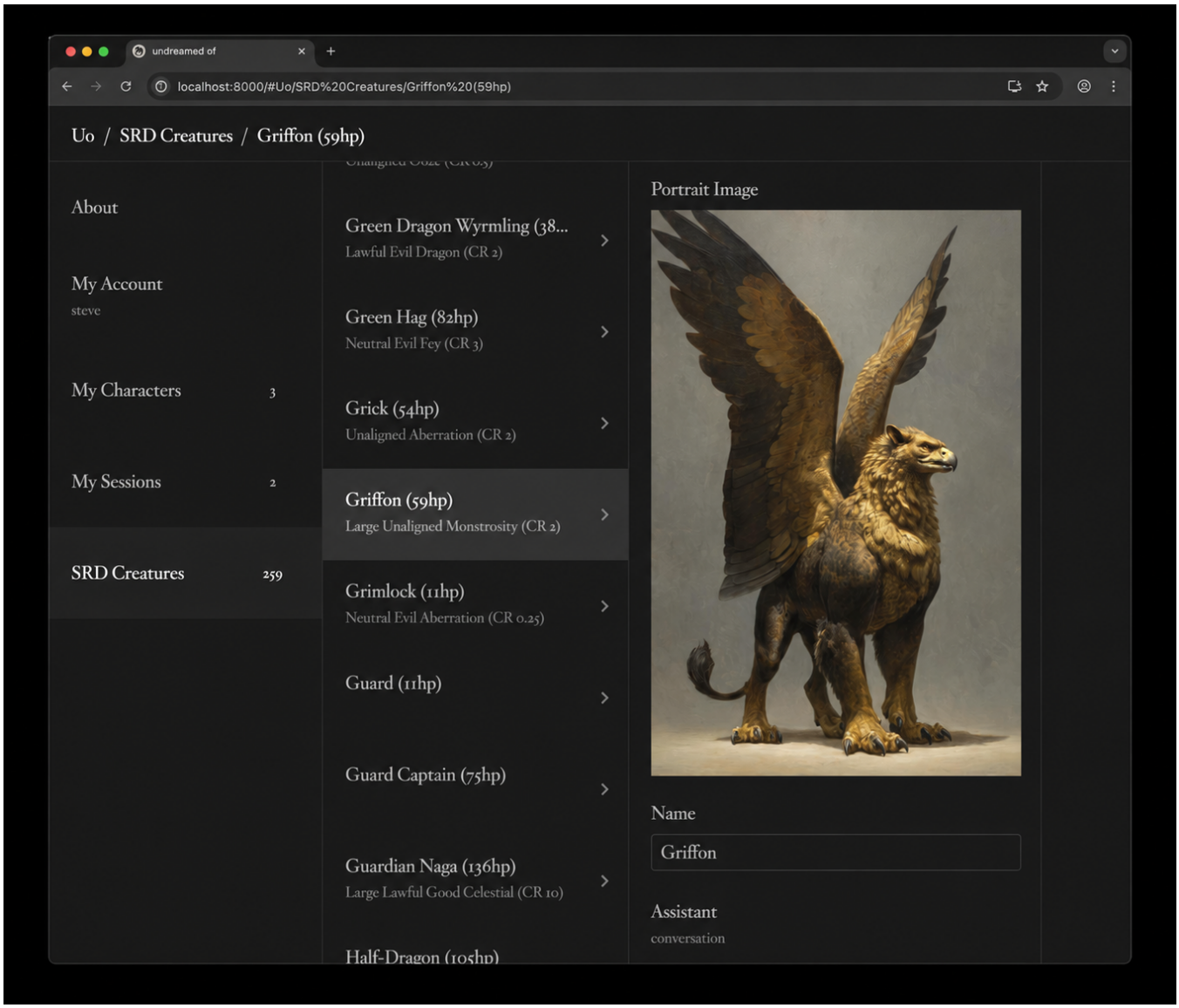



**Figure 14.** undreamedof.ai built with Strvct.

| Dimension | Traditional Component Frameworks | Low-Code / Visual Builders | Strvct (Modern Naked Objects) |
|---|---|---|---|
| Primary Artifact | Authored Component Trees | Visual Layout Diagrams | Annotated Domain Model |
| UI Derivation | Manual per screen | Manual per screen | 100% Derived automatically |
| Model/UI Coupling | High (manual props & state) | High (visual bindings) | Zero (model has no UI code) |
| Maintenance Cost | Scales with screen count | Scales with screen count | Near constant (model-driven) |
| AI Agent Interop | Per-screen tool specs | Custom API connectors | Native constant-size tool surface |
| Offline & Cloud Sync | Complex manual sync logic | Server-dependent | Built-in local pools & versioned commits |

**Table 3.** Comparison of UI Development Paradigms.